



Software Tools 复习整理 — 来自 softwaretool.docx

使用说明

- 这份笔记按考试高频主题重排，方便快速查找。
- 页码默认指向 index777.pdf 的 PDF 页码（如 p.9）。
- 涉及真题时，会标注 tb2-example-answers.pdf 的对应考点。

Week 01 - HTTP 与协议基础

1) 请求消息 vs 响应消息 — p.4

- 请求 — Request：客户端发给服务器，包含方法、路径、请求头等。
- 响应 — Response：服务器返回，包含状态码、响应头、响应体。
- 考试常见点：看清“是否有响应体”、状态码含义、头字段用途。

HTTP 方法与幂等性 — 讲义 1.7.3 + GET/POST/PUT/DELETE

方法	做什么 — 概括	幂等？ — 课程口径	讲义常举
GET	读资源，不应改服务器状态 — 理想语义	✓ 幂等	多次 GET /user/123 → 照样取信息，资源不按“多读几次就多改几次”累积变化
POST	提交数据：创建资源、触发处理；实务里也常承载自定义写操作	✗ 通常非幂等	POST /user 每发一次常新建一条，结果可不同

方法	做什么（概括）	幂等？（课程口径）	讲义常举
PUT	整份新表示替换目标 URI 上资源（或 API 约定下“没有则建”）	✓ 幂等（同 URI + 同 body 重复提交，最终状态视为一致）	<code>PUT /user/123 + JSON</code> ；常与 Content-Type / Content-Length 同框
DELETE	删除目标资源	✓ 幂等	<code>DELETE /user/123</code> ；已删之后再删 → 不应再累积改变状态。实务上可有权限制

实务注意：POST 顶替 DELETE / PUT（index777 1.7.3）

现实情况	常见替代写法
许多服务器 不实现或忽略 DELETE / PUT	用 POST + 自定义语义：URL 查询、路径片段或请求体里的操作字段
例（删除）	<code>POST /user/123?action=delete HTTP/1.1</code> 表示删掉用户 123 ，但方法仍是 POST

考场分辨：考题问 **RFC/REST** 标准语义 → 删用 **DELETE**，全量替换用 **PUT**；问 为啥线上看见 `POST?action=delete` → 答 兼容性/网关只放行 **POST** / 服务端未启用 **PUT·DELETE**。

常见请求头一眼表（以 `GET /about.html` 为例，index777 1.7.2）

头字段	含义（概括）
<code>GET /about.html HTTP/1.1</code>	请求行：方法、 request-target 、HTTP 版本
<code>Host: www.example.com</code>	目标主机名（HTTP/1.1 下与路径一起定资源）
<code>User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:108.0) Firefox/108.0</code>	浏览器 + 环境（此处 Firefox 108 跑在 Linux x86_64 上）

头字段	含义（概括）
	Linux 64 位 + X11 ； Mozilla/5.0 只是历史通用前缀 ‘≠’ → Mozilla 操作系统
Accept: ...	客户端愿意要的响应类型（HTML / XHTML / XML 等）
Accept-Language: en-GB,en;q=0.5	语言偏好 ‘首选英式英语’ 普通英语权重较低（q）
Accept-Encoding: gzip, deflate, br	客户端能解压的压缩格式（br=Brotli）；表示内容编码协商 ‘不是’ → 用户在终端对网页执行 gzip 命令
Connection: keep-alive	希望本连接在响应后尽量复用 TCP ‘供同一客户端后续请求用’ ；不是 ‘多个浏览器同时发同一请求’

模拟题 Q2（ index777 p.5 ）：从请求推断 ‘谁’ 访问了什么

```
GET /about.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:108.0) Firefox/108.0
Accept: text/html,application/xhtml+xml,application/xml
Accept-Language: en-GB,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
```

假设请求头可信 → 答案 ‘B. 有人在 X11 图形环境下用 Firefox 浏览
http://www.example.com/about.html （ GET + Host + User-Agent 一致 ）。

选项	判定
A	X 把 Accept-Encoding: gzip 误读成 ‘在终端对网页跑 gzip’ ；头只表示可接受压缩响应。
C	X Mozilla OS 与 UA 不符 ；关键信息在

选项	判定
	Firefox/108.0 + (X11; Linux ...) °
D	X Connection: keep-alive 是单客户端连接复用 ’不是多个浏览器同时逼服务器保活 °

同类题 : **curl + hello.txt** ㄣ **resit** / 卷面变体 ㄣ

```
GET /hello.txt HTTP/1.1
User-Agent: curl/7.16.3 libcurl/7.16.3 ...
Host: www.example.com
Accept-Language: en, mi
```

答案 : **B**. 使用 **curl** 访问 **http://www.example.com/hello.txt** ㄣ 路径 + Host + User-Agent: curl ㄣ °

选项	为什么错
A	无响应 `无状态码` ’不能断 5xx °
C	Safari UA 与 curl 矛盾 °
D	请求行 + Host 合法 ’不宜说 ㄣ客户端格式错误ㄣ °

一句话背题 : **User-Agent** 认软件栈 ; **GET + path + Host** 认资源 ; **Accept-* / Connection** 不要过度故事化 °

对齐 **index777** **p.2-5** ` **tb2** **Q1/Q2** 类 HTTP 报文阅读 °

2) HTTP 状态码五大类 ㄣ **p.9** ㄣ

- **1xx** 信息提示
- **2xx** 成功 ㄣ如 **200** , **204** ㄣ
- **3xx** 重定向
- **4xx** 客户端错误 ㄣ如 **400** , **404** ㄣ
- **5xx** 服务器错误

对应真题：状态码首位判断题（如 Q3）

3) fetch Promise 的状态（p.9）

- `fetch(url)` 的 Promise 目标是“拿到一个 Response 对象”。
- 只要收到了 HTTP 响应（即使是 404 / 500），Promise 仍是 **fulfilled**。
- 只有网络层失败（DNS 连不上、协议错误等）才是 **rejected**。

4) Q22：TCP 在第几层 OSI？（tb2 Q22）

题干：HTTP 在 OSI 第 7 层（Application），TCP 在哪一层？

答案：C. 第 4 层 Transport（传输层）。

选项速判

选项	层	对不对
A 6 Presentation	表示层 / 编码与抽象	✗ TCP 不在此
B 5 Session	会话层（理论偏多）	✗
C 4 Transport	端到端：端口、可靠/分段	✓ TCP、UDP
D 2 Data Link	链路：帧、MAC、同一段链路	✗

讲义已给的栈（和 OSI 对照背）：`HTTP -> TLS -> TCP -> IP`。OSI：HTTP≈应用层（7），TCP≈传输层（4），IP≈网络层（3）；TLS 常画在 TCP 之上，教材对 5/6 层的细分不必和考试抬杠——本题只认准 **TCP=L4**。

OSI 七层一览（职责 + 协议举例：讲义口径；与同文件「必备知识」表同源）

层号	OSI（中英）	这层做什么（讲义口径）	常见协议 / 关键词
7	Application 应用层	直接为用户 / 应用程序	HTTP / HTTPS、FTP、DN

层号	OSI(中英)	这层做什么(讲义口径)	常见协议 / 关键词
		提供可用的网络通信(报文语义、具体应用怎么用)	S ；亦常见 SMTP 等
6	Presentation 表示层	数据格式的转换与协商；加密/解密、编码等——怎么表示——的问题	TLS (也常画在 TCP 之上的 HTTPS 栈里)；JPEG 等格式例
5	Session 会话层	管理通信会话：建立、维护、终止连接/对话状态	SOCKS (代理/会话代表例)；单题考得少
4	Transport 传输层	端到端传输(常理解为进程到进程)；可靠性、流量控制、端口	TCP (可靠、面向连接)、 UDP (无连接)；tb2 Q22 TCP 在本层 ✓
3	Network 网络层	路由与转发：把分组/数据包投递到对岸网络；逻辑寻址跨多个网络	IP (v4/v6)、 ICMP (常与 IP 同记)
2	Data Link 数据链路层	在同一段链路/LAN 上传帧；链路节点之间的可检测错/寻址—— MAC	以太网(IEEE 802.3 MAC 子层)、 Wi-Fi (802.11 MAC)； MAC 地址
1	Physical 物理层	比特在介质上的电	Ethernet

层号	OSI (中英)	这层做什么 (讲义口径)	常见协议 / 关键词
		气/光学/ 射频传输 (一) 线上 是什么波形 (一)	PHY (802.3 物理层) ` 双绞线/ 光纤 ` 无线电 ` 蓝 牙物理层等

(上表与同文件 必备知识笔记 → HTTP / 网络基础 内 **tb2 Q22** 表 完全一致 ' 任选一处改动时
请同步两处)

Week 02 - HTML

1) HTML Elements 基础 (p.20)

- 重点是识别合法元素与语义用途 (结构含义 ' 而不是外观)
- 题目常见陷阱 : 看起来像属性名/值的词并不是元素名

2) 模拟题 Q4 : URI 里的 fragment (#...) (tb2 / URL 常与 HTML 一起看)

答案 : **D**. Fragment 是对**同一已取回资源内部的子部分 (或次级位置)**的可选引用 ' 不是整段 URI 的 规范说明书

典型写法 : `https://example.com/page#section2`

- `#section2` 即 fragment → 常用于 锚点 : 浏览器滚到文档内 `id="section2"` 等处 (不向服务器换一个不同 path)

URI 几个部分别混在一起 (讲义思路)

URI 段落	起首	题干里常被描述成	不是.....
path	/ ... (在 host	资源在服务上的	fragment

URI 段落	起首	题干里常被描述成	不是.....
	后面~	层级标识~如 /books/fiction/ ~	
query	?	key=value 参数映射~可选~	fragment
fragment	#~在末尾最常见~	可选 '指向同一资源内部的某段子资源/位置	~URI 规范的引用~ (A) `层级 path (B) `query (C)

选项速判~你的解析概括~

选项	为什么错
A	像在说是 RFC 等规范文档 '不是 URI 自身结构里的 fragment 。
B	那是 path~层级路由~ '不是 # 。
C	那是 ?query 键值参数 '不是 fragment 。

与 HTTP 的注意点 : 发给服务器的 request-target 一般只含 path?query '不包含 #fragment ~fragment 通常只在客户端处理~ 。

3) 模拟题 Q5 : <a> 里哪一段是浏览器里 ~可点的链接文字~ ~ index777 p.25 ~

答案 : B. <a>... 内部的文本内容~锚点元素的 content '不是整条段落 '也不是属性值~ 。

解题思路~不展开具体题干~

1. ~点链接~时 '视觉上高亮可点的通常是 起始标签 <a> 与结束标签 之间那一段文字 。

2. **href** : 目标地址 ; 题干若把 **URL** 字符串当成 “你要点的字” 经常是干扰项 °
3. **title** : 辅助说明 / 悬停提示 ; 不是你当作链接主体去点的那个短语的典型答案 °
4. **p** 里的其它文字 : 在 **a** 外的前后句 , 不参与当成 “这一条超链接的标签文字” °

4) Spring Boot + Thymeleaf (实验补充 : 动态页面与 index777 里 Thymeleaf 题干一起看)

名词	一句话
Spring Boot	基于 Java 的 Web 快速开发框架 , 少配置 , 快起可运行项目 (Spring 生态的 “上手套装”) °
Thymeleaf	服务端 HTML 模板引擎 : 后端把数据塞进模板 , 页面里用表达式把数据渲染成最终 HTML °

合在一起做什么

- 动态 **Web** : 从数据库取数 → 放进 **Model / Context** → 渲染模板 → 浏览器拿到完整 **HTML** °
- 模板能力 (记维度即可) : 遍历 , 动态属性 , 动态文本 °
- 动态 **URL** : **Controller** + 路径变量 , 常见 **@PathVariable** : 从 URL 片段取参数 (如 `/.../{id}`) °

请求到页面的一条链 (背顺序)

1. 用户访问 **URL**
2. **Spring Controller** 处理
3. (按需) 数据库取数据
4. 构造 **Context / Model**
5. **Thymeleaf** 渲染模板
6. 返回 **HTML** 给浏览器

分工口诀 : **Controller** = 接到请求 , 取数 , 组模型 ; **Thymeleaf** = 把模型填进 HTML 再输出 °

5) resit Q28 : @GetMapping("/student/ids") 能推出什么 ? — tb2 / Thymeleaf 题干同类 —

答案 : D. None of the above.

解题思路 — 只记逻辑 —

1. `@GetMapping("...")` : 把 HTTP GET 映射到引号里那一串路径 ; `/student/ids` 是合法路径 '不是 — 路径写错所以永远不会进 Controller — (C X) °
2. `/student/ids` 里的 `ids` 是字面路径段 '不是 `{id}` 这种路径参数占位符 → 不能仅凭这一行就断定 — 方法接受某个 student ID 路径变量 — (A X) ° 要绑 URI 片段 '通常写成 `/student/{id}` + 方法参数上 `@PathVariable` °
3. 返回值 : 映射注解只描述路由与 HTTP 方法 '不描述返回的是 ID `视图名还是 JSON — 不能从题干这句单独推出 — 返回 student ID — (B X) °

选项	判定
A	X 把 <code>ids</code> 误当成路径参数 ; 动态段要 <code>{...}</code> °
B	X 返回值需看方法签名/返回类型 '不能由 path 单独断定 °
C	X 映射本身合法 °
D	✓ 以上都不足以从注解一条推出 °

Week 03 - CSS 选择器与盒模型

1) 选择器核心 — p.32 —

- `#jumble` : 匹配 `id="jumble"`
- `.jumble` : 匹配 `class="jumble"`
- 空格 : 后代选择器 — 任意层级 —
- `>` : 子选择器 — 仅直接子元素 —

2) 组合器速记 — p.33 —

- 后代选择器：A B
- 子选择器：A > B
- 相邻兄弟：A + B —紧挨着—
- 通用兄弟：A ~ B —后续同级—

3) 讲义 5.7.2 CSS 选择器与组合器知识点总结 —同源 表：index777 §5.7.1 '标题 5.7.2' PDF 约 p.36–37 —

说明：PDF 中 5.7.2 为 —知识点总结— 小节名；各行组合器条文出自紧挨着的 §5.7.1 —阅读器脚注常见 -- 36 of 103-- ~ -- 37 of 103-- 一带—。

选择器 vs 组合器 —讲义一句—

- **Selector**：要选中的元素模式。
- **Combinator**：两个选择器在 DOM 树上的位置关系 —空格、>、+、~ 等—。

组合器一览 —课堂总表 | 摘自 index777 §5.7—

名称 —中英文—	写法	匹配关系 —记这个—
后代 Descendant	A B —A 与 B 之间 空格—	B 在 A 内部 '任意层级后裔
子代 Child	A > B	B 仅是 A 的直接子元素
通用兄弟 Subsequent-sibling	A ~ B	同父、在 A 之后出现的所有兄弟 B —不必挨着—
相邻兄弟 Next-sibling	A + B	同父、B 紧挨在 A 后面的那一个兄弟
列 Column	双竖线：col 与 td 两选择器之间写两个竖线 — ASCII U+007C x2—	与表格 column 范围相关 —了解即可—
命名空间 Namespace	单竖线：如 SVG 与 a	限定 XML/SVG

名称（中英文）	写法	匹配关系（记这个）
	之间写一个竖线（配合 <code>@namespace</code> ）	等命名空间（了解即可）

与 §2 速记的对照：考试主力仍是 空格 / > / + / ~ ；**列组合器（双竖线）与命名空间（单竖线）**讲义放在 5.7.1 后部，可作了解（MDN 见讲义 §5.7 链接）。

4) 典型题：form+button, .stray

- 逗号 , 表示“选择器列表”，两边独立生效。
- form+button 表示“紧跟在 form 后面的 button”，不是 form 内部的 button。
- .stray 匹配所有 class 为 stray 的元素。

5) Box Model / Margin（p.40）

字号与长度单位（讲义例：html { font-size: 12pt; }、p / h1 / h2）

单位	相对／绝对	讲义要点（记基准 + 本例算式）
pt	绝对	Point ，印刷常用 ；html { font-size: 12pt; } ⇒ 根元素字号 12pt 。
rem（root em）	相对根（:root / html）	1rem = html 上声明的 font-size 。本例 ；p { font-size: 1rem; } ⇒ 段落 12pt ；h1 { font-size: 1.8rem; } ⇒ 1.8 × 12pt = 21.6pt ；h2 { font-size: 1.4rem; } ⇒ 16.8pt 。
ex	相对当前所用字体	约等于该字体里**小写 x

单位	相对／绝对	讲义要点（记基准 + 本例算式）
		**的视觉高度，随字形而变。 ° 本例 : <code>margin-top: 2ex</code> ⇒ 2 倍 x 高 ； <code>margin-bottom: 1ex</code> ⇒ 1 倍 x 高 。

- **em vs rem**（一句）： **em** 相对本元素最终算出来的 `font-size`； **rem** 只跟 **html** 根字号则全文档里用 **rem** 的字号一起缩放。
- **Reset**（一句）：常把 **html / body** 浏览器默认的外边距、内边距清掉，与「从零开始排版」同属 **CSS reset** 常见动机。
- **<h1>** 块级： **h1** 是块级，默认可铺满包容块宽度；讲义里 `width: 100%` 是显式写满 **body**（或父容器）宽度，与默认块级盒行为一致。
- **inline** 与 **block** 的行为差异是常考点（见下方 §5.1 展开）。
- **margin collapse** 关注「垂直相邻外边距合并」。

5.1 inline vs block 行为差异（展开）

一句话记忆

- **block**（块级盒）：自己占一整行，可设宽/高，四边 **margin/padding** 都按盒模型生效，垂直方向 **margin** 会 **collapse**。
- **inline**（行内盒）：跟着文字流走，宽/高被忽略（`width/height` 不生效），只有左右 **margin** 影响布局，上下 **margin** 基本不推开周围元素；上下 **padding** 视觉上会鼓出来但不撑开行高，容易视觉重叠。
- **inline-block**（折中款）：像 **inline** 一样并排，但像 **block** 一样接受 `width/height` 与四边 **margin/padding**（参见 §5 末尾的左右盒示例 `display: inline-block`，行 304）。

对照表（高频考点全集中在这一张）

维度	block	inline	inline-block
是否独占一行	是（前后自动换行）	否（跟文字一起排）	否（并排在一行）

维度	block	inline	inline-block
<code>width / height</code>	生效	忽略（非替换元素）	生效
左右 <code>margin / padding</code>	生效	生效（会推开同行内容）	生效
上下 <code>margin</code>	生效，且参与 vertical collapse	不参与布局（一般不推开上下行）	生效，不参与 collapse （自身建立独立格式上下文）
上下 <code>padding</code>	生效，撑开盒高	背景会鼓出，但不撑开行框，可能视觉压到相邻行	生效，撑开盒高
默认排布方向	垂直堆叠	水平随文字流	水平随文字流
典型默认标签	<code>div / p / h1-h6 / section / span / a / em / strong</code>		<code>需手动设 display: inline-block</code>

替换元素例外：img / input / video / iframe 默认是 **inline-replaced**（行内替换），虽然是 inline，但 **width/height** 仍然生效——这是讲义里容易踩的坑。

速判口诀（考试现场用）

- 看到题目让一设 `width` 给 `<span>` ——默认 inline，宽度无效，需要先 `display: block` 或 `inline-block`。
- 看到题目让两个块水平并排 ——选 **inline-block**（或 flex / grid，但讲义这章先用 inline-block 的多）。
- 看到上下垂直外边距合并 ——只在 **block** 之间发生，**inline** 不参与，**inline-block** 也不参与（它建立 BFC）。
- 看到左右两个 inline 之间空隙是两边 margin 之和 ——对照 §5 第 315 行的左右并排示例。
- 看到 `<h1>` 默认 100% 宽 ——**block** 行为，参考行 269 的笔记。

小例子 1：宽度对 `<span>` 不生效

```

<style >
  span.a { width: 200px; height: 50px; background: #fcc; }
  span.b { display: inline-block; width: 200px; height: 50px; background:
#fcc; }
</style >
<span class="a"> A 宽高被忽略 </span >
<span class="b"> B 宽高生效 </span >

```

- `span.a` : '宽高被忽略' 盒子大小完全跟文字走 °
- `span.b` : '改成 `inline-block` 后' `200×50` 才真正生效 °

小例子 2 : `inline` 的上下 `margin/padding` 不推开行

```

<style >
  span.big { padding: 20px 0; margin: 30px 0; background: #def; }
</style >
<p > 前一行文字 <span class="big"> 行内大块 </span > 后续文字。 </p >
<p > 下一段文字仍然紧贴上面。 </p >

```

- `span.big` 的背景会因为 `padding: 20px 0` 鼓出来 '但行高没有被撑开' 会视觉重叠上下文字 °
- `margin: 30px 0` 在 `inline` 上对垂直布局没意义 '下一段不会被往下推' °

小例子 3 : `block` vs `inline-block` 的 '占行' 差别

```

<style >
  .blk { display: block; width: 120px; height: 40px; background: #fdd; }
  .ib { display: inline-block; width: 120px; height: 40px; background: #dfd; }
</style >
<div class="blk"> 块1 </div > <div class="blk"> 块2 </div > <!-- 上下堆 -->
<span class="ib"> A </span > <span class="ib"> B </span > <!-- 左右排 -->

```

- 两个 `.blk` 各占一行 '垂直堆叠' °
- 两个 `.ib` 并排在一行 '且 `width/height` 都生效' °

和 §5 已有内容的衔接

- §5 第 273–297 行讲 **margin collapse** ——只在 `└block + 垂直相邻┐` 时发生；`inline / inline-block` 都不参与。
- §5 第 299–316 行的左右盒示例正是 `display: inline-block`，所以中间空隙是 `20 + 20 ≈ 40px` 相加，而不是 collapse 取较大值。
- 行 269 写 `└<h1>` 块级默认可铺满包容块宽度——也是 **block** 行为的直观体现。

Q11 (tb2)：Margin collapse 是什么意思？

答案：A

思路

- `margin collapse` 出现在正常文档流里，常见是两个块级元素在垂直方向上相邻时，上下的 `margin` 会合并，不是简单相加。
- 合并结果通常类似取较大的那一侧（复杂情况还有父子参与合并，本题选项只考了典型描述）。
- B/C/D 描述的是别的现象（`inline` 左右 `margin`、视口自适应、负数失败模式），都不是 `margin collapse` 的标准定义。

选项速判

- **A**：✓ 垂直相邻、`margin` “碰在一起”、剩较大的、较小的在合并语义里不参与相加。
- **B**：✗ 把话题扯到 `inline` 的左右 `margin`，不对题。
- **C**：✗ `margin` 不会因此被 viewport “动态缩放”成这样。
- **D**：✗ 不是 collapse 的定义；负 `margin` 是另一回事。

小例子

```
<style >
.a { margin-bottom: 30px; }
.b { margin-top: 20px; }
</style >
<div class="a"> 上 </div >
<div class="b"> 下 </div >
```

两框之间的空隙接近 `max(30px, 20px) = 30px`，而不是 50px。具体是否完全等同 `max` 要结合是否同一 BFC，考试记“垂直合并、不单纯相加”。

对比：左右并排时，中间空隙常是「两边 margin 相加」

```
<style>
.box {
  display: inline-block;
  width: 80px;
  height: 40px;
  background: #ddd;
}
.left { margin-right: 20px; }
.right { margin-left: 20px; }
</style>
<span class="box left"> 左 </span> <span class="box right"> 右 </span>
```

- 左盒右缘到右盒左缘之间，常见是 $20\text{px} + 20\text{px} \approx 40\text{px}$ 这种“两格 margin 一起算”的间距。
- 这和上下块级相邻的 **margin collapse**（不直接 $30+20$ ）是不同场景。

6) Media Query 做题流程（p.40）

1. 先判定媒体类型：screen 还是 print
2. 过滤不匹配的 media 条件（宽度、设备等）
3. 再看选择器优先级与继承
4. 最后判断最终样式（常见是颜色）

7) Q10：@media print vs 屏幕宽度 vs print {} 陷阱（tb2 Q10）

考点（4点）

1. 打印时用 **print** 媒体，不是浏览器当前视口宽度：@media screen ... 在打印时整体不生效。
2. **print { color: black; }** 是元素选择器，只有 HTML 里真有 `<print>...</print>` 这类标签才匹配，不是“打印样式”，别和 **@media print { ... }** 搞混。
3. 继承：h1 没有单独在 **@media print** 里设 **color** 时，会继承父级（常见是 **body**）的 **color**。
4. 层叠：打印时 **@media print** 里对 **body** 的声明会覆盖外层 **body { color: blue; }**。

Q10 解题（Jack 打印）

1. 打印 ⇒ 忽略所有 `@media screen and (...)` （不管 700px `600px` `800px`）。
2. `print { ... }` 不作用在 `h1`，除非页面上有 `<print>` 元素包裹 `h1`（题目没有）。
3. `@media print { body { color: yellow; } }` 生效 ⇒ `body` 为黄色。
4. `h1` 未单独设颜色 ⇒ 继承 `body` ⇒ 黄色。答案 C。

易混例子 1：误以为屏幕宽度影响打印

```
@media screen and (max-width: 600px) { h1 { color: red; } }  
@media print { body { color: yellow; } }
```

说明：打印时只有第二段有效；`red` 不会出现（除非另有非 `screen` 的规则给 `h1`）。

易混例子 2：`print` 标签 vs 打印媒体

```
print { color: black; } /* 选中 <print> 元素 */  
@media print { body { color: green; } }
```

说明：打印整张纸时 `body` 变绿；除非你写了 `<print>h1 here</print>`，否则 `print { }` 碰不到普通 `h1`。

易混例子 3：打印时专门改 `h1`

```
body { color: blue; }  
@media print {  
  body { color: yellow; }  
  h1 { color: red; }  
}
```

说明：打印时 `h1` 用 `red`（比继承 `body` 更具体）。

易混例子 4：只有全局 `body`，打印无额外规则

```
body { color: blue; }  
@media screen and (min-width: 800px) { h1 { color: black; } }
```

说明：打印时没有 `@media print`，`@media screen` 不应用；`h1` 仍继承 `body` ⇒ 蓝色。

8) CSS 覆盖顺序（层叠）速查

做题时按下面顺序想，‘靠前的先筛掉不适用的规则’再在剩下的里比谁赢。

第 1 步：这条规则当前会不会参与？

- `@media screen { ... }` 在打印时不参与；`@media print { ... }` 在屏幕上不参与。
- 宽度条件（`max-width` / `min-width`）只对匹配该媒体的那段有效；别用“当前窗口 700px”去推打印结果。

```
h1 { color: navy; }
@media screen and (max-width: 600px) { h1 { color: red; } }
@media print { h1 { color: green; } }
```

- 屏幕、窗口 500px：red 生效（满足 `screen` + `max-width`），不是 green。
- 屏幕、窗口 900px：两段 media 都不命中宽度，h1 用 navy。
- 打印：不管窗口多宽，`@media screen` 整块不参与 ⇒ h1 是 green。

第 2 步：直接声明 vs 继承

- 元素上有对应属性的声明（如 `h1 { color: red; }`）优先于从父级继承来的值。
- 没有自己的声明时，才沿父级链条继承。

```
body { color: gray; }
article { color: purple; } /* article 自己是 purple */
h1 { color: red; } /* h1 有直接声明 */
p { } /* p 没有 color, 继承父级 */
```

```
<body >
  <article >
    <h1 > 标题 </h1 >
    <p > 段落 </p >
  </article >
</body >
```

- `h1` : 红色 (`h1 { color: red; }`) 不沿用 `article` 或 `body` 的继承色。
- `p` : 没有 `p` 的 `color` $\Rightarrow$ 继承最近祖先 `article` 的 **purple** (不是 `body` 的 `gray`)。

第 3 步 : 选择器特异性 (specificity)

- 同一天平上比 : 一般 **#id** > **.class** > 类型选择器 (同类型再比“写了几个”) 如 `article h1` vs `h1`。
- 特异性高的规则赢 (与在文件里谁先写无关 (特异不同阶时))。

```
h1 { color: blue; } /* 1 个类型 */
.title { color: orange; } /* 1 个 class : 通常压过单独的 h1 */
#site-title { color: black; } /* 1 个 id : 通常压过 .title */
```

```
<h1 id="site-title" class="title"> 站名 </h1>
```

- 最终 **color: black** (`#site-title`) 赢。

再一例 (类型选择器“越长”通常越具体) :

```
article h1 { color: green; } /* article + h1, 两个类型指向同元素 */
h1 { color: orange; }
```

```
<article> <h1> 标题 </h1> </article>
```

- `h1` 同时匹配两条 , 但 **article h1** 特异性更高 $\Rightarrow$ **green** (若两则权重算平 , 才落到第 4 步比谁后写)。

第 4 步 : 特异性相同 $\rightarrow$ 看谁后写

- 同源 (同特异性时) 后出现的覆盖先出现的。

```
body { color: blue; }
body { color: brown; }
```

- 屏幕上看 `body` : **brown** (后写)。

打印 + `body` 两条都进池子：

```
body { color: blue; }  
@media print { body { color: yellow; } }
```

- 打印时 `body` 以 **yellow** 对 `body` 的打印规则在后面，且作用于 `print` 场景下与 `blue` 竞争时，按层叠后者常赢；两则都是 `body` 单选择器时后写者优先。

第 5 步（了解即可）：内联、`!important`

```
<p class="note" style="color: pink;"> 内联 </p >
```

```
.note { color: brown; }
```

- 一般 `style="color: pink"` 比 `.note` 更硬 $\Rightarrow$ 段落是 **pink**。

```
.note { color: brown !important; }
```

- 作者样式里加了 `!important` 时，常会压过没有 `!important` 的内联；再往上还有用户 `!important`，UA 规则，考试很少抠到那么深，知道“多一层优先级”即可。

一页口诀

1. 媒体：screen / print 先分流。
2. 直连胜继承：元素自己有 `color` 就不用先看 `body`。
3. 特异性：谁更具体谁赢。
4. 再平手比顺序：写在后面的赢。

Week 04 - CSS 布局进阶

1) Grid Layout（p.50）

- 自动放置（auto-placement）沿文档书写方向推进（通常左到右、上到下）。

- 常见错误：误以为新项总放同一格，或忽略 `writing mode`。

2) Q12：Grid auto-placement tb2 Q12

答案：B

思路

- 多个未指定确切 **grid** 位置的子项会用 **auto-placement** 依次摆上网格。
- 算法按文档的 **writing mode**（书写方向 + 块的流向）去找最早能放得下的格子，不是题干里编的“preference rank”。
- A 说成“topmost leftmost”只覆盖常见英文横排情况，没提 **writing mode**，在课程表述里不如 B 严谨。
- C 错：每个新项不会都挤在“第一行第一列”同一格。
- D 错：没有这种标准“preference rank”考点属性。

可视化：九宫格（3 列 × 若干行）`列三等份 1fr`

把容器宽分成 3 等份：`grid-template-columns: 1fr 1fr 1fr` 或 `repeat(3, 1fr)`。

5 个子项依次 auto-place（LTR 常见顺序）在长这样铺满：

	列1 (1fr)	列2 (1fr)	列3 (1fr)
行1	①	②	③
行2	④	⑤	(空)

- 口诀：先顺书写方向填满“一行里够早的空位”，再换行。不是 5 个都挤在同一个格子里（否定选项 C）。

演示 1：代码对应上图（三等列 + 自动摆 5 项）

```

<style >
.g {
  display: grid;
  /* 三列等宽 = 宽度的三份 */
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 48px;
  gap: 8px;
  max-width: 360px;
}
.g > div { background: #cde; border: 2px solid #468; text-align: center;
line-height: 48px; }
</style >
<div class="g">
  <div > 1 </div > <div > 2 </div > <div > 3 </div > <div > 4 </div > <div > 5 </div >
</div >

```

- 视觉上就是上面 ASCII：'1-3' 在第一行，'4-5' 在第二行前两格。顺序由 **writing mode** 下的“最早能放的位置”决定，所以考试表述选 **B**。

若题目想强调行也高矮均分：在容器有固定高度时可用

`grid-template-rows: repeat(2, 1fr)`（两行各占一半高）与上面的
`repeat(3, 1fr)`（三列各占三分之一宽）是一套思路：行列都可以用 **fr** 做等份。

可视化：某项占两列宽（跨两轨），后面几项接着找空

	轨1	轨2	轨3	轨4
	1fr	1fr	1fr	1fr
行1	A (col span 2)	B	C	
行2	D	...		

演示 2：某一项占多列，后续项会绕开已占区域找下一个够大的空位

```

<style >
.g2 {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 40px;
  gap: 6px;
  max-width: 400px;
}
.g2 > div { background: #fec; border: 2px solid #a60; text-align: center;
line-height: 40px; }
.wide { grid-column: span 2; }
</style >
<div class="g2">
  <div class="wide"> A </div >
  <div > B </div >
  <div > C </div >
  <div > D </div >
</div >

```

- A 占两轨后，同一行还剩两格给 B、C，D 到下一行左端继续排；仍是按 **writing mode** 扫描“够大且够早”的空。

演示 3：为何提 **writing mode**（概念示意）

/* 仅作概念：竖排或 RTL 时，“先扫哪一格”会与纯英文 LTR 不同 */

- 考试记一句：自动放置顺序跟文档 **writing mode** 走，所以标准表述用 **B** 而不是死写“永远左上角”。

Week 05 - JavaScript 基础

1) 字符串拼接陷阱（p.56）

- 字符串与数字用 `+` 时，会发生字符串拼接。
- 例如：`14 + "5"` 结果是 `"145"`，不是 `19`。

2) Q13 : `input.value` 、 `0xE` 与 `+` (tb2 Q13)

答案 : D — 输出 `Now it's 145`。

知识点

- `input.value` 是字符串 ; `0xE = 14` ; `14 + "5"` 按拼接得 `"145"`。
- 对应笔记 : `index777.pdf` p.56-60 (JS 基础 类型与运算符)。

3) Q14 : 谁不是 JavaScript 关键字 (tb2 Q14)

答案 : C. `not`

思路

- `this` : 关键字 '指当前执行上下文里的 `this` 绑定 (方法调用 严格模式等课里会讲)。
- `of` : 在 `for (x of iterable)` 里是关键字 (`for...of` 遍历)。
- `let` : 关键字 '声明块级作用域变量'。
- `not` : 不是关键字 ; 逻辑非用运算符 `!` '不是英文单词 `not`'。

对应 `index777.pdf` (便于回去翻讲义)

项	在笔记里的大概位置
<code>let</code> 变量与作用域	p.56, 57, 60, 63, 64 等
<code>for...of</code> 、 <code>of</code>	p.57
关键字 reserved / keyword 相关表述	** p.57, 60 ' 及 p.66, 68 (偏异步章节也会用到关键字语境)
<code>!</code> 与非 (<code>not</code>)	.logic 一般用 <code>!expr</code> ; 讲义里别把英文 <code>not</code> 当成关键字

必备区分

- 关键字：语言保留，不能用作变量名（如 `let a = 1;`）。
- `not`：自然语言；在 JS 里写否定请用 `!`、`!==` 等。

例子

```
let ok = true;
if (!ok) { /* 合法：! 是运算符 */ }

// for...of 里的 of 是语法一部分
for (const v of [1, 2, 3]) { console.log(v); }
```

4) Q15：let 禁止在同一作用域重复声明（tb2 Q15）

答案：D. 控制台 `a`、`b` 都不会打印（Neither）。

思路（与本题一致的执行顺序）

1. `if (curval > 20)`：`curval === 20` 条件为假 → 不打印 `a`。
2. `let curval = curval + 1`：顶层已有一个 `let curval`，再在同一脚本顶层块声明同名 `let` → 语法错误（Duplicate / already been declared）。
3. 一旦在这里触发错误，后面的 `if ... console.log('b')` 根本不会执行成功 → `b` 也不会出现。（考试按讲义结论选 两者都不出现）

和 `index777.pdf` Week 05 的对应知识点（模拟题常客）

讲义 p.56（变量与 `var` / `let`）明确对比：

- `var`：旧式声明，在同一作用域内允许重复声明，容易埋 bug。
- `let` / `const`：现代推荐，不能像 `var` 那样在同一作用域重复声明同名变量——这正是本题陷阱。

本题等于考：别把第二行写成又一个 `let`，只能赋值。

正确改写（只想 +1）

```
let curval = 20;
if (curval > 20) console.log('a');
curval = curval + 1; // 或 curval += 1 / curval++ (只赋值, 不要第二个 let)
if (curval > 20) console.log('b'); // curval === 21 → 会打印 b
```

对照记忆—讲义里的两条“模拟题向量”

讲义位置	考点
p.56	var vs let / const ; 重复声明 ; 推荐用 let
p.57	分支 if ; 比较别写错— > vs >= —
p.56–57	数字与字符串 ` + — 可与 Q13 一起复习—

5) index777 JavaScript—Week 05—其余高频考点查漏补缺

下面是你笔记里 **Software.md** 已通过 Q13–Q15 `字符串`、`let / var` 覆盖过的以外，讲义里还经常出题或做选择题的块—页码均为 **index777.pdf PDF 页**—。

语言与设计—约 p.56 —

- 解释型—课件 + 模拟：`interpreted`，不是纯传统意义的“compiled-first”。
- 动态类型：变量无静态类型声明—`typeof` 可用来查运行时类型—。
- 与 **HTML/CSS** 分工：结构 / 表现 / 行为—**JS**—三层递进。

数字与运算符—约 p.56 ‘Lab p.62 —

- 只有一种 **number**：内部IEEE双精度；`0x` 十六进制、`0b` 二进制。
- 常见运算：`+` `-` `*` `/` `%`、`++`、`--`、`**` 幂。
- 边界：`1/0` → `Infinity`，`-1/0` → `-Infinity`—了解即可—。

字符串—约 p.57 —

- 单引号 / 双引号 / 反引号模板字符串：`...\${expr}...`。
- 已强调：`"100" + 1` → `"1001"`—与 Q13 同源—。

分支与比较—约 p.57 —

- `===` 严格相等（推荐）vs `==`（会隐式转换，易错）。
- `if / else if / else`；`switch`：`case`、`break`、`default`（漏 `break` 会贯穿）。

循环（约 p.57）

- `for`、`for...of`（遍历可迭代对象，与关键字 `of` 相关 → 连 Q14）。
- `while / do...while`；`break / continue`。
- 讲义提到数组上常用 `map / filter`（知道是“返回新数组/筛选”即可）。

函数（约 p.57-58）

- `function` 声明；箭头函数 `const f = (a,b) => a+b`。
- 讲义例：`push`、模板字符串、`join` 组合（和数组方法一起考读代码输出）。

对象（约 p.58 'Lab p.63-64）

- 对象字面量、`const` 绑定的对象引用 vs 属性可改。
- 点号 / 方括号访问；嵌套对象。
- **JSON**：`JSON.stringify / JSON.parse`；函数不会被序列化进 **JSON**（解析后方法变 `undefined`）—— 易出理解题。

数组（约 p.60、p.63 Lab）

- 索引从 0；`length`、`push / pop`、`unshift`、`indexOf`、`includes`。
- `sort()` 会改原数组；`toSorted()` 得新数组（讲义对比）。
- `splice`：注意 `indexOf` 得 -1 时 `splice(-1,1)` 会误删最后一项（讲义标成陷阱）。

DOM 与事件（约 p.58-62）

- **DOM** 是树；`document`；`document.body`。
- 查询：`getElementById`、`querySelector`、`querySelectorAll`。
- 易考点：`querySelectorAll` 返回静态列表；`getElementsByClassName` 返回动态（live）集合。
- 创建/插入：`createElement`、`createTextNode`、`appendChild`、`insertBefore`。
- 内容：`textContent`（纯文本，更安全）vs `innerHTML`（解析 HTML，XSS 风险）。
- 样式：`element.style` 是行内样式；`classList.add/remove/toggle` 优于硬改 `className`。
- 事件：`addEventListener` 可绑多个；HTML `onclick` / 单个 `.onclick=` 易覆盖、可维护性差。

- 脚本太早执行：`getElementById` 得 `null` → `<script>` 放底部、`defer`、`DOMContentLoaded` / `window.onload`。

Node 类型（约 p.61）

- 元素节点、文本节点、注释节点；`nodeType` 数值（讲义列举 1/3/8/9）——可能出“是什么类型”题。

Node / 命令行（Lab p.62 起）

- `console.log`：无 `return` 时函数结果为 `undefined`。
- 本块与 Q16–Q17（Promise / fetch）在 p.65+ Week 07 与 Week 05 分开复习即可。

Week 07 - 异步 JavaScript

1) Q17：Promise.all 何时调用 .then 里的回调 （tb2 Q17，p.67）

```
Promise.all([promise1, promise2, promise3]).then(commonHandler);
```

答案：B. `commonHandler` 至多调用一次，且仅在三个 **Promise** 全部都 **fulfilled** 之后。

选项怎么排

- **A x**：不是“谁先 fulfilled 谁先触发一次”；`Promise.all` 不会为每个子 Promise 各调一遍 `then`。
- **B ✓**：全部成功 ⇒ 合成的 Promise fulfilled ⇒ `then(commonHandler)` 跑 0~1 次（这里就是恰好 1 次）。
- **C x**：不是三次；且 **rejected** 的子 Promise 会让 `Promise.all` 整体 **rejected**，根本不会进这个 `then`。
- **D x**：若任一子 Promise **rejected**，`Promise.all` 立马 **rejected**，`.then(commonHandler)` 不会执行——不是“不管 fulfilled/rejected 都等齐再调一次”。

口诀

- `all` = 全真才兑现；一票否决就走 `catch`（或末尾的 `rejection` 处理）。
- `commonHandler`：一次批处理结果，不叫“轮询触发”。

讲义

- `index777.pdf` p.67（Week 07 JavaScript 阅读材料，`Promise.all` 语义）。

2) Q16：fetch 与 404 时 Promise 状态（tb2 Q16）

答案：B. `fulfilled`

思路

- `fetch(url)` 的 Promise 只保证一件事：尽量完成一次 HTTP 请求并拿到 `Response`。
- 404 / 500 仍然是成功送达的响应 → Promise `fulfilled`；在 `.then(res => ...)` 里再看 `res.ok`、`res.status` 处理业务失败。
- `rejected`：通常是拿不到响应（网络错误、DNS、CORS 导致失败、`TypeError` 等），别把 HTTP 错误状态码当成 Promise `rejected`。

和 `index777` 对齐

- 与 p.66–67（异步、`fetch`/网络请求解析）一起看；也与 Week 01 里 `fetch Promise` 小节（p.9 话术）同一考点。

伪代码（易混点）

```
fetch('/typo-path') // 404
  .then(r => { /* 会进来 */ }) // fulfilled
  .catch(e => { /* 404 通常不进来 */ });
```

Week 08 - Web Scraping 与输入校验

本章小节顺序：§1–2 = 输入校验（与同周讲义里 表单、`fetch`、`preventDefault` 同属，别信客户端）；§3–5 = 爬虫（**tb2 Q19** `wget` → Q20 行为规范 → Q21 BS4）内容

Q18 开头的题号顺序一致；若只关心爬虫可跳读 §3 起。

1) Q18：防“过长密码”最重要做什么 — tb2 Q18 —

场景：限制密码长度以保护数据库 — 避免极端长输入、buffer/性能/存储滥用等。

答案：**D. Server-side input validation**

为什么 D

- 写进数据库之前的最后一道闸门在服务器。攻击者可以绕过浏览器：改前端、关掉 JS、curl / Postman 直接 POST、抓包重放。只有服务端在看到请求体时再量一次长度，才防得住恶意客户端。

A / B / C 算什么

选项	作用	能当安全主线吗
A polite notice	提示用户，可有可无	X
B maxlength / HTML constraint	浏览器侧提示+限制，可被绕过	X
C JS submit 监听器检查长度	仍是用户机器上执行的规则	X

讲义对应

- 课程 **JavaScript Week 02** / 给定阅读：强调 客户端校验不能替代服务端；本题与 讲义 + 过往卷同源。
- index777 Week 08 侧重里既有 爬虫 / wget，也有 p.70 左右的表单 preventDefault + fetch；校验谁说了算仍是 服务端——下文 爬虫小节里 — 不要相信客户端发来的数据——是同一条安全总纲的另一面。

2) 与 validation 相关的其它高频点 — 顺带背 —

一句话原则

- **UX** / 少用流量：前端 — HTML / JS — 可以早拒绝、早提示。
- **安全 / 数据完整**：涉及存库、权限、钱的规则 → 服务端必须再验一遍。

HTML5 表单约束（浏览器辅助，可被绕过）（知道名字即可）

- `required`、`maxlength` / `minlength`、`pattern`（正则）、`type`（如 `email`，`number`）`min` / `max` / `step` 等。
- 可用 JS：`input.checkValidity()`、`reportValidity()`（仍是客户端语境）。

JS 校验（仍为客户端）

- `submit` / `change` / `blur` 上检查再在 UI 报错 → 改善体验；不能直接当安全边界。

纵深防御（defense in depth）

- 允许：前端校验 + 服务端校验都做；不允许：只有前端就认为安全。

和本节 Q18 / 下文爬虫（wget / LAB）的关系

- 你在 p.70 一类练习里自己做 表单 + `preventDefault` + `fetch`：若服务端不校验参数，就只信任浏览器里的脚本——Q18 考这道坎；爬虫链路则是——别假设 HTTP 另一端按你从网页上看到的规矩来。

3) 爬虫：wget 与同目录二次下载——记忆表（讲义 p.71、p.74 Lab；tb2 Q19）

先背：同一目录、同一目标文件名，第二次下载会发生什么——由 默认 / `-nc` / `-N` 决定；题干四个选项正好是三种真行为 + 一个干扰项。

情境	wget 怎么干	和 Q19 选项关系
默认（不加 <code>-nc</code> ）	本地已有同名文件时，常为保留第一份，新来的自动改名 <code>*.1</code> 、 <code>*.2</code> ...	对应错题 B（默认行为，不是 <code>-nc</code> ）
<code>-nc</code> （no-clobber）	不覆盖：已有同名则跳过 / 不重下，不会出现 <code>.1</code> 备选名	Q19 正解 ✓ C
<code>-N</code> （timestamping）	看 远端 vs	对应错题 D

情境	wget 怎么干	和 Q19 选项关系
	本地时间戳：远端更新才重下，可能覆盖等条件行为	
直接覆盖第一份	wget 常规「防误覆盖」不是这样 （题干把「无脑覆盖」当干扰）	对应错题 A

tb2 Q19 一句话

- 第二次下载 `\*\*nc`：第一次文件留着，wget 拒绝再下第二份（到同名路径）°→ C °

讲义里常与「爬虫」一并记的参数（索引 **index777 ~ p.71-74**）

参数	英文名/概念	背什么
-p	page requisites	拉 HTML + 依赖资源（CSS/JS/图等），单页离线更像浏览器里看到的那样
-r	recursive	递归跟链接往下抓
-l N	level=N	递归深度上限（=1 只多跳一层外链）
-m	mirror	整站镜像（讲义：等价一串默认组合，离线备份常用）
-np	no-parent	不向「父目录 URL 」回溯（常与递归一起用来控范围）

根据讲义实验（**cattax / localhost:8080**）想定：页面上有什么，命令会爬到什么

假设站点结构类似：index.html 里用 `<link href="catstyle.css">` 引样式，用

`<a href="Felinae.html">` / `Pantherinae.html` 链到两内页；还有全站 `robots.txt` ° 与 `index777` `p.74` 左右 **Lab** 描述一致 °

你敲的命令	大致会下载到什么	题里常考点
<code>wget .../index.html</code> 〰 无额外参数	只有 <code>index.html</code> 这一个文件	本地打开常缺 CSS/图 ；因为没跟 HTML 里引用的静态资源
<code>wget -p .../index.html</code>	<code>index.html</code> + 它所依赖的资源 〰 讲义例： <code>catstyle.css</code> 等 page requisites ；实验里还常连带拿到 <code>robots.txt</code>	〰 单页离线看起来像浏览器 〰 要 <code>-p</code>
<code>wget -r -l 1 .../index.html</code>	从首页再跟出 〰 直接链接到的 〰 那一层页面 〰 例： <code>Felinae.html</code> 、 <code>Pantherinae.html</code> ，以及实验里提到的 <code>robots.txt</code> 等	<code>l=1</code> = 递归深度 1 〰 只多跳一层链接
同上但不加 <code>-p</code>	有子页 HTML ，但若子页也引 CSS/图 ，这些不会因 <code>-r</code> 单独就全下齐	讲义原话：递归不一定带齐资源，要整页展示常再叠 <code>-p</code>
<code>wget -m ...</code> 〰 mirror	按镜像规则 尽量复制整站 〰 范围、深度由组合决定，比 〰 只 <code>-r -l 1</code> 大得多	考 〰 备份整站 〰 时想 <code>-m</code>
<code>-np</code> 合用	在子路径开抓时，不往 〰 父路径 〰 上爬 〰 防一下爬到整站根上一层去	与 <code>-r</code> 一起用来 缩范围

记忆句：只点一个 **URL** → 多半只有那一页；要 〰 像网页一样全 〰 → `-p`；要 〰 跟链接继续

走_└ → `-r` + 用 `-l` 控层数；要_└别往上爬出界_└ → `-np`。

模拟选择题（_└会抓下来什么_└ 题型_└）

以下假设备份站都在 同一主机：`http://lab.local/`，且 `robots.txt` 不禁止。

共有页面（_└题里就这些_└）：

- `/index.html`：内部有 `<link href="catstyle.css">`，还有 `<a href="a.html">`、`<a href="b.html">`。
- `/a.html`：只有 `<a href="c.html">`（从 `index` 要再进一层才到 `c`）。
- `/b.html`、`/c.html`、`/catstyle.css` 各为独立文件。

题 1. 执行：`wget http://lab.local/index.html`（不加 `-p`、不加 `-r`）_└一定会被拉下来的是？_└

- A. 只有 `index.html`
- B. `index.html` 和 `catstyle.css`
- C. `index.html`、`a.html`、`b.html`

答案：A。单 URL 单次请求，默认不自动跟 `<link>` / `<a>`。

题 2. 执行：`wget -p http://lab.local/index.html`，在“page requisites”含义下，至少多出来哪类内容相对题 1？

- A. 仍会只有 `index.html`
- B. 会有 `index.html` + `catstyle.css`（及同页其它依赖资源）
- C. 会把 `a.html`、`b.html` 也抓下来

答案：B。`-p` 跟的是该页展示依赖（CSS/JS/图等），不是递归跟所有 `<a href>` 子页面。

题 3. 执行：`wget -r -l 1 http://lab.local/index.html`（不加 `-p`）_└一般会多下哪些？_└

- A. 只有 `index.html`
- B. `index.html`、`a.html`、`b.html`（有时还会带上 `robots.txt` 等，视服务器与配置）
- C. 连 `c.html` 也会下（因为站上存在链）

答案：B。`-l 1` = 从起点再跟链接 1 层：只含 `index` 直接指向的 `a.html`、`b.html`。

`c.html` 在 `a` 的下一层 ’要 `-l 2` (或更大) 才可能抓到。

同题型再练：根据 **HTML / 站点结构** 判断 会下哪些 / 共几个文件

判题速记 (递归默认行为 ’题干未另写 `-H` 等开关时)

你要看的	记住
只看起点 URL	成功则至少 1 个资源 (常与 1 个文件对应)。
<code>-r -l 1</code> 且 不加 <code>-p</code>	会跟 本页 <code></code> 超链接再往下一层 ; 不会仅因 <code><link></code> / <code></code> / <code><script></code> 就把 CSS `图` 脚本 (像递归 <code><a></code> 那样) 全拉下来。
加 <code>-p</code>	对已决定要下的 HTML , 再拉 page requisites (该页渲染依赖的 CSS/ 图等) ; 与 <code>-r</code> 跟谁 是两套规则 (见上题 2)。
<code>-l</code> 的层	起始页上的 <code><a></code> 算第一层延伸 ; 仅在子页里才出现的链接 , 多数要 <code>-l 2</code> 才会继续跟。
外链主机	<code>-r</code> 默认常在同一主机内 ; 题干写 <code>https://other.com/...</code> 时 , 一般不会当成本次递归的一部分。
不产生新 HTML 文件	<code>#fragment</code> 、 <code>mailto:</code> 、 <code>javascript:</code> 一般不增加 (多下一个 <code>.html</code> 这类计数)。
<code>robots.txt</code>	递归时 可能被访问并保存 ; 题目若选项差 1 个 , 要读题干是否把 robots 算进 (下载文件数) (讲义常说 (有时还会带上))。

变式题 4 对标： `wget -r -l 1 http://example.com/` '无 `-p`'
设首页 / 常为 `index.html` 只有：

```
<link rel="stylesheet" href="/site.css">
<a href="/p1.html"> ... </a>
<a href="/p2.html"> ... </a>
```

则常考点：会下几个「页面级」文件「先不算 `robots`」 $\Rightarrow$ 3 个「首页 + `p1.html` + `p2.html`」。`site.css` 仅在 `<link>` 里出现 '无 `-p`' $\Rightarrow$ 不算在本命令「自动跟出来的」那一批里。

变式题 5 同首页加 `-p`

`wget -r -l 1 -p http://example.com/`：在题 4 基础上至少多出 `site.css`「首页 `requisites`」；子页是否还带齐各自 CSS/图「依 `-p` 对多层页面的语义」考场抓主旨：「`-p` 管依赖链」不替代「要不要跟进 `<a>` 子页」这件事「那是 `-r -l` 管的」。

变式题 6 外链干扰

首页既有 `<a href="/local.html">` '又有 `<a href="https://not-example.com/x.html">` 执行 `wget -r -l 1 http://example.com/` $\Rightarrow$ 多起跳 同站的 `local.html`；不要选「外站 `x.html`」也会被本次递归拉下。

变式题 7 深度同题 3

首页只有 `<a href="a.html">`，`a.html` 内才有 `<a href="c.html">`。`-r -l 1` 从首页 $\Rightarrow$ `c.html` 不会下；要到 `c` 需 `-l  $\geq$  2`「或从 `a.html` 单独起一条 `wget`」。

变式题 8 计数口算

首页有 `k` 个同站「可解析的 `<a href>`」无外链「无其它特殊说明」。`-r -l 1` 无 `-p` $\Rightarrow$ 常说约 `1 + k` 个 HTML「再视题干是否计入 `robots`」。

题外对比「讲义常出现」：`wget` vs `curl` —— `wget` 偏向 递归/整页/镜像下载；`curl` 偏向 单次请求/API/表单调试。「细节以讲义表格为准」。

4) Q20 : 爬虫 / 抓取要遵守什么 (哪条不算 guideline) (tb2 Q20)

题干 : Guidelines ... DO NOT include → 答案 : D

D 为什么错 (考场逻辑)

- “访问网页”和“在本机保存一份拷贝”在技术上是同一路径的请求/响应 : 浏览器展示页面本身就会把内容拉到你机器上。不能简单笼统地说“可以随意访问”但下载就必然侵权——这是 把水搅浑，不是爬虫课教的规范。
- 版权 / 再利用要讲 用途、许可、合理使用、robots/TOS : 见 正确项 C，而不是 D 这种说法。

A / B / C : 都是合理 guideline (要选“不包括”就选 D)

选项	要点 (背一句)
A	能走 官方 API / 数据源就少爬 HTML，省事，也少踩限流/条款。
B	遵守 robots.txt (以及站点条款) ; DISALLOW 的路径不要乱撞。
C	即使内容是 公开的，转载/再发布仍可能受版权、隐私、许可证约束，不能想当然。

与本章其它题的关系

- §1 Q18 : 安全上 服务端说了算 ; §3 Q19 = 工具 ; §4 (本题) = 行为规范 ; §5 Q21 = 解析 HTML。爬虫 能干 ≠ 该干。技术能下 ≠ 道德/法律上可以随便做。

5) Q21 : BeautifulSoup 取 “第一条链接” 的文案 (tb2 Q21)

答案 : C. soup.main.div.div.a.text

逐项拆 (考场常坑)

选项	实际得到什么	为什么不合题
A <code>soup.find_all('a')[0]</code>	Tag 第一个 <code><a></code> 元素 不是 str	题干问 字符串 "Page 1" → 一般要 <code>.text</code> / <code>.get_text()</code> / <code>.string</code> 视子结点而定
B <code>soup.find('l1').a.text</code>	<code>find('l1')</code> 找 标签名 <code><l1></code> '文档里没有' → 多半是 None '再 <code>.a</code> 会报错	应用 <code>find(id='l1')</code> 、 <code>soup.select('#l1 a')</code> 等按 id 定位
C	main 下第一个 <code><div></code> → <code>.container</code> '其下第一个 <code><div></code> → <code>#l1</code> '里边 <code>.a</code> ' <code>.text</code> 拼出可视文本 Page 1	属性式链式：每一步都是该结点下同名标签的第一个
D <code>...footer.parent.find('div').f</code> <code><a> l1</code> 但 <code>.contents</code> <code>ntents</code>	<code>[1-1]==0</code> 确实指第一个 返回的是 list 子结点列表 不是裸字符串 "Page 1"	若写 <code>.text</code> / 索引 <code>[0]</code> 再 str 才可能贴近题意

顺带记：题干里的 **Python** 小细节

- `[1-1]` 就是 `[0]` 考你能不能看出 表达式当索引 仍是合法下标。
- `find_*` / `select` 的定位对象：标签名、`id`、`class`、`attrs` 等；别把 `id="l1"` 当成标签名 `l1`。

同源具象：本题 **HTML** 上 写什么 → 会返回什么 页面骨架：`main` → `p.info` → `div.container` → 五个 `div#l1...l5` '各内含一个 `a href="./page?.html"`。

术语先对齐 背概念名 + 题干里怎么用

说法	BeautifulSoup / 本题里怎么用
Tag	元素结点 例如某段

说法	BeautifulSoup / 本题里怎么用
	<code><div>...</div></code> 、 <code><a>...</code> 〰 题干里 <code>find_all('a')[0]</code> 就是这样一个对象 不是 Python <code>str</code> 。
<code>NavigableString</code>	文本片段的包一层类型 常与 <code>Tag</code> 一起出现在 <code>.contents</code> 的列表里 。 <code>.string</code> 在 结构简单 时往往指向这一段 。
<code>None</code>	<code>find(...)</code> 未命中的返回值 。 <code>soup.find('l1')</code> 即此 要找的是 <code>name='l1'</code> 标签名片段 <code>l1</code> 〰 文档里没有 <code><l1></code> 只有 <code>id="l1"</code> 的 <code><div></code> 。
<code>find_all</code>	返回 <code>list</code> ；逐项仍是 <code>Tag</code> 〰 常见情况 〰 本题 <code>find_all('a')[5]</code> 会越界 〰 <code>[0]</code> 才是 <code>Page 1</code> 那一个 。
<code>.text</code> / <code>.get_text()</code>	从当前 <code>Tag</code> 的子树里抽出 可读成一段话的字符串 ； 对空白折叠等与 <code>.contents</code> 机械串联不完全相同 〰 本题 <code><a></code> 内只有字面 <code>Page 1</code> → <code>.text</code> 结果为 <code>'Page 1'</code> 。
<code>.string</code>	仅 内部结构简单 时好用 ；本题 <code><a>Page 1</code> 只有纯文本 → 常得到 <code>NavigableString</code> ；若 <code><a></code> 里再嵌 <code></code> 〰 <code>**string</code> 常为 <code>None</code> 〰 要用 <code>.text</code> 。
<code>.contents</code>	仅 直接子结点 构成的 <code>list</code> 〰 <code>NavigableString</code> ∪ <code>Tag</code> 混合 〰 类型永远不是题干要的裸

说法	BeautifulSoup / 本题里怎么用
	<code>str</code> ；选项 D 输在这一点 °
<code>tag.div</code> / <code>tag.a</code> ￣快捷属性访问￣	从当前结点 向下找 文档顺序里第一个同名标签 °这是 traverse API ’不是 CSS 那种 ￣必选第几项子结点￣ °
<code>select_one</code> / <code>select</code>	走 <code>#id</code> ` <code>class</code> ` <code>></code> ` 空格 ￣后代￣ 等 CSS 选择器语义 °

本题里 ；表达式 → 类型 / 形状 → 和 **"Page 1"**

你写的 ￣与试题同源￣	主要返回值	"Page 1" ￣ <code>str</code> ￣ ?
<code>soup.find_all('a')[0]</code>	Tag	否 °要接上 <code>.text</code> °
<code>soup.find_all('a')[0].text</code>	str	是 → 'Page 1' °
<code>soup.find_all('a')[0].content</code>	list ￣本题常 一项 <code>NavigableString</code> ￣	否 °是 序列 ’不是题干字符串 °
<code>soup.find('l1')</code>	None	—— find 的参数是标签名的 <code>name</code> ’不等于 <code>id</code> °
<code>soup.main.div.div.a.text</code>	str ￣经 <code>.text</code> ￣	是 °即 选项 C °
<code>soup.find('footer').parent.find('div').find_all('a')[1-1]</code>		否 ° <code>on **[1-1] **= [0]</code> 指对了 第一个 <code><a></code> ’但最后一步 <code>.contents</code> 仍然是 列表 ￣选项 D￣ °

链式 `.div.div` 为何本题恰好命中 ； `main` 的子树里 第一个 `<div> = container` ’再在 `container` 里第一个 `<div> = #l1` °若在 `main` 里 ` `p` 之前多插一个没有语义用途的空 `<div>` ’第一段 `.div` 会先黏在那个空壳上 ’整链就错位——这跟 `select('#l1 a')` 仍能指到 `#l1` 的稳定度不同 °

同一目标拿到 **Page 1** 的「更稳写法」对照

代码	术语上讲「你在用什么机制
<code>soup.select_one('#l1 a')</code>	CSS 选择器 ：按 id 属性 + 后代 a 定位，不依赖「第几个 .div 」。
<code>soup.find(id='l1').find('a')</code> 或等价按属性过滤	find(**kwargs) 按 HTML 属性（ id / class_ ）筛选，再走子 a 。
<code>soup.find('main').find('div', class_='container').find('a', id='l1').a.text</code>	组合 find 和 id 属性， id='l1' 属性收窄，每一步意图比裸 .div.div 更透明。

与同周 **wget** 的分工「放一起背」

	wget	BeautifulSoup
处理对象	URL ↔ 磁盘文件（HTTP 应答体落成字节/文件）。	str / bytes 形式的 HTML ， parser 建好文档树。
典型动词	下载、镜像、 <code>-r -l -p</code> 。	find / select / .text 抽字段。
一句话	「弄到本地」。	「已经读到内存里了，再在树上走路」。

先有 抓取到的 **HTML**（`wget`、`requests.get().text`、`open().read()` 等），再做 `BeautifulSoup(html, 'html.parser')`，本题里的 **soup...** 才有意义。

Week 09 - 密码学 / 证书 / 邮件安全

1) 证书链字段阅读

- **s** (subject)：证书属于谁

- **i** (issuer) : 谁签发
- **NotAfter** : 过期时间

2) Q23 : Public Key Cryptography 是什么 — **tb2 Q23** —

答案 : **C** — 两把钥匙 : 一把公钥公开 , 一把私钥保密 — 。

错项速判

- **A** : 说成 — 只有一把 key 再加密码 — → 不是非对称加密定义 。
- **B** : 两把都保密 → 这不是 public-key 体系 。
- **D** : 只有一把公开 key → 缺少 private key , 不成立 。

必背 : 公钥/私钥两种典型用途

目标	发送方做什么	接收方做什么	结论
保密 — Encryption —	用接收者公钥加密	用接收者私钥解密	只有私钥持有者能读
身份+完整性 — Signature —	用发送者私钥签名	用发送者公钥验签	证明是谁发的且内容未改

结合 **index777** — **Week 09 p.79-85** — 的应用场景

- 证书链/HTTPS : 证书里带网站公钥 ; 读证书常看 **subject** 、 **issuer** 、 **NotAfter** — 与本章 §1 对应 — 。
- **PGP**/邮件加密 : 发邮件给 Bob 时 , 用 **Bob** 公钥加密 , Bob 用 **Bob** 私钥解密 — (保密性) — 。
- 邮件签名 : Alice 用 **Alice** 私钥签名 , 收件人用 **Alice** 公钥验签 — (身份 + 完整性) — 。
- 签名 + 加密一起做 : 同时得到机密性 、 可认证性 、 完整性 — (Week 09 高频组合) — 。
- **Git** — **SSH** — 认证 : 把 公钥 放到 GitHub/GitLab 账号 , 本机保管 私钥 ; **git push/pull** 时服务器用公钥验证你用私钥完成的签名挑战 — (证明“你是你”) — 。
- **Git** 提交签名 — **GPG/SSH signing** — : 提交由私钥签名 , 其他人用你的公钥验签 ; 用于确认提交来源与完整性 — (不是给仓库内容做保密加密) — 。

3) 为什么“签名 + 加密”邮件

- 加密 : 保证内容机密性 — (他人不能读) —

- 签名：验证发送者身份，并保证内容未被篡改

Week 10 - RFC 与网络工具

1) 一屏总览（index777 p.90-94）

术语	一句话定义	和谁最容易混淆
RFC	互联网规范文档体系（Request For Comments）	不是组织；它是“文档”
IETF	推动互联网标准制定的组织/社区	不是文档；它是“人和流程”
Protocol	通信规则（格式、时序、状态、错误处理）	不是实现代码
Socket	OS 提供的网络通信端点抽象	不是具体协议名
Port	主机内服务入口编号（区分进程）	不是 IP 地址
inetd	按需拉起网络服务的 super-server 机制	不是新网络协议

2) 关系图（考试最常考）

- IETF 产出/维护标准过程，RFC 承载标准文本。
- Protocol 是规则本体，程序通过 socket 使用这些规则通信。
- 连接定位可理解为：IP 定位主机，port 定位主机内服务。
- inetd 管理的是“服务如何被唤起”，不改变协议定义本身。

3) 高频易错对照

容易说错	正确说法
RFC 是组织	RFC 是文档体系 ; 组织是 IETF
端口在网络层	端口概念主要跟传输层 (TCP/UDP) 关联
Socket 就是 TCP/UDP	Socket 是编程接口抽象 ; TCP/UDP 是传输协议
inetd 是协议	inetd 是服务管理/分发机制

4) Q27 : Berkeley Socket 一句话定性 (tb2 Q27 '对应 index777 p.91 一带)

最优定义 : **Socket = network endpoint** (网络连接端点) 。

为什么不是 C (file descriptor)

- 在 Unix 实作里 'socket 常以 **file descriptor** 形式操作 (可 read/write 或 send/recv) 。
- 但题目问“Berkeley Socket 是什么” ’课程语义更看重概念本体 : 它是“端点抽象” ’不是“FD 这个实现细节本身” 。

反向对齐 index777 的同块证据 (Week 10)

- socket() / bind() / listen() / accept() 这一串调用 (p.91) 都在描述“端点建立 `监听` 接入” 。
- inetd 章节强调 : 连接到来时把 socket 交给服务处理 (stdin/stdout 重定向语境) 。
- /etc/services 与端口映射 (如 finger 79 `gopher 70) 体现“端点 + 端口 + 协议”三者配合 。

5) 速记卡 (考前 20 秒)

- RFC = Request For Comments** (Q26 直接考点) 。
- Berkeley Socket = 端点 (endpoint)** ’不是“某个大学项目” `不是 DNS 解析器 。
- IETF:** 组织 ; **RFC:** 文档 ; **Protocol:** 规则 ; **Socket:** 端点 ; **Port:** 服务入口 ; **inetd:** 按需拉起服务 。

- 混合题先拆成 6 个名词的“角色” ’再匹配选项 °

Week 11 - Testing **index777** Slides **p.97–100** • Lab 文档 **p.101–103** ∩

1) 讲义在讲什么 ∩ 先立总纲 ∩

- 测试目的 : 不是追求数学上“绝对正确” ’而是弄清代码何时 `为何` 怎样失败 ∩ QA 也同理 ∩ °
- 单点正确示例不等于全对 : `mean()` 一例可过 `换输入即暴露 整除截断` `printf` 格式错用 `累加溢出` 等 ∩ 讲义均值例子 `p.97` 一带 ∩ °

2) ∩ 测试武器库 ∩ 一屏对照 ∩ 与 **index777** 21.3 节同序 ∩

方法	一句话	典型工具/讲义提法	和 Q29 fuzz 关系
Assert	开发期插条件 `违例就停`	<code><assert.h></code> ; <code>-DNDEBUG</code> 可整段删掉	不是 fuzz ; 是程序员假设自测
Unit test	已知输入 → 已知期望输出	Python : <code>pytest</code> ∩ 讲义示例 ∩	不是 fuzz ; 是样例回归
Behavioural / BDD	自然语言描述行为 `再映射成代码`	Cucumber ` <code>.feature</code> + step definitions	不是 fuzz
Property testing	随机大量输入 `验某条性质/ 不变量始终成立`	QuickCheck ∩ Has kell ∩ ; 失败会 shrink 成小反例	与 fuzz 都大量随机 `但性质测试要自己想 property
Fuzz testing	随机或半随机 `含损坏数据灌进接口 `找 crash /	讲义提 AFL++ `分支覆盖 思路	tb2 Q29 : 选 ∩ 随机输入直到崩溃/ 异常 ∩ 

方法	一句话	典型工具/讲义提法	和 Q29 fuzz 关系
	未定义行为 / 安全洞		
Formal proof	数学证明全体输入下满足规范	Coq `Isabelle `Lean	代价最高 ; 不是 fuzz

3) Q29 : Fuzz testing 唯一定位 — tb2 Q29 —

- 正解 — 概念 — : 通过 随机/畸形输入 摸索边界 , 直到出现 崩溃 、异常 、内存错误 等 → **B** 。
- 易混 : **A** = 单元测试 — 对预期行为的用例 — ; **C** = 图形渲染无关 ; **D** = 形式化证明 , 不是 fuzz 。

4) Q30 : 性质测试 — QuickCheck 类 — 相对传统单元测的优势 — tb2 Q30 , index777 p.99–101 —

答案 : **C** — 能 自动生成大量测试用例 , 不必像单元测那样 逐个手写 (输入, 期望输出) 。

index777 里怎么定义 **Property testing** — 死记这段关系 —

- 不是测 — 一个具体输入的正确输出 — , 而是 : 对随机生成的大量输入 , 检验某条 **property** — 性质/不变量 — 是否总成立 — 讲义 21.3.5 , p.99 — 。
- **QuickCheck** — Haskell 经典 — : 自动生成数据 → 跑性质 → 失败时 **shrink** — 缩小 — 到最小反例 , 便于定位边界 bug — 讲义 21.3.6 , p.99–100 — 。
- 讲义例子 : 性质写得“太想当然”会在 整数溢出 等边上炸出来 — 如 $b^e > b$ 一类 , p.100 文本 — 。
- 代价/缺点 — 讲义原话方向 — : 你得自己想出值得测的 **property** — 这不是免思考的银弹 — 见 21.3.5 — 缺点 : 需要自己想 property — — 。

选项速判 — 结合讲义 —

选项	判定
A	x 仍要用代码写 property 与生成逻辑 ; 不是 — 不会写代码也能测 — 。
B	x

选项	判定
	─更快─ 不一定成立；随机+多轮有时更慢 °优势在 覆盖面/用例数量，不绑在 wall-clock °
C	✓ 对应 自动生成大量用例，免手写每一条 °
D	✗ 全体输入证明 是 Formal proof ；性质测试是抽样/随机探索，不是数学证明 °

与 Unit test 的一行对比 ─ p.98-99、Lab p.101 ─

- **Unit**：一组 已知好/已知坏 的输入与断言 ─pytest 等─ °
- **Property**：一条 普遍命题 + 框架批量随机验证；Python 侧讲义常对齐 **Hypothesis**，Haskell 侧 **QuickCheck** ─Lab 推荐表─ °

5) index777 里和 Q29 同周，必须一起背的点

- **Property testing vs Fuzz** ─ p.99-100 ─：前者验 **property** ─如交换律、可逆性─；fuzz 更偏 探路找崩，安全场景多 °
- 回归 ─**Regression**─：以前过的测试现在挂了 = 改动引入退步 ─讲义 21.4.1─ °
- 为什么测试不能保证 **100%**：无法枚举一切输入/组合；过测试 ≠ 设计对 ─讲义 21.4.2─ °
- **Lab 推荐栈** ─ p.101 起─：单元测 **pytest / unittest**；性质测 **Hypothesis** ─Python─、**QuickCheck** ─Haskell─等——与目录里 ─fuzz / property / QuickCheck─一致 °
- 同文件后部 ─ p.102 ─：云计算 vs 雾/边缘 ─易与 Q33 干扰项混─；**HTTP** 写在 **IETF RFC** ——与 Week 01/10 可串 °

6) 考前 15 秒 ─Week 11─

- **Fuzz** = 随机灌入 + 盯 crash/安全；**Unit** = 固定用例 + 期望输出；**Property** = 随机 + 验一条命题；**Formal** = 证明，不是测 °
- **Q30**：QuickCheck 类优势 = 少手写用例、多自动探索；≠ 更快、≠ 不需会写代码、≠ 证明全输入正确 °

考前速记（超短版）

- `fetch` 收到 404 仍是 fulfilled；网络失败才 rejected。
- `Promise.all`：全 fulfilled 才 then；有一个 rejected 就走 catch。
- CSS 选择器先看符号：空格（后代）、>（子代）、+（相邻兄弟）、~（通用兄弟）。
- 安全校验必须在服务端。
- BeautifulSoup：`find('l1')` ≠ `id="l1"`；`.contents` ≠ 字符串；`.text` 才是常用（可见文案）。
- 状态码先看首位：2 成功、3 重定向、4 客户端错、5 服务端错。读请求看 `User-Agent`（`Mozilla/5.0` ≠ Mozilla OS）；`Accept-Encoding` ≠ 终端跑 gzip；`keep-alive` ≠ 多浏览器并发；`GET + path + Host` 定资源（`index777 p.2-5` 模拟 Q2）。
- URI：`?query` = 参数；`#fragment` = 同资源内部锚点（模拟 Q4/ Week 02 §2）；HTTP request-target 通常无 #。
- `a` 链接：可点的可见文字在 锚点标签对之间（`<a>...</a>` 的内容）；≠ 整段 `p`，≠ `href` 里的 URL（模拟 Q5 'Week 02 §3'）。
- Spring：`Controller` 处理 URL/取数；`Thymeleaf` 渲染 HTML（Week 02 §4）。`@GetMapping` 只标路由；`{id}` 才是路径变量（Q28）。
- TCP = OSI 第 4 层（Transport）；HTTP = 第 7 层（Application）。七层表：必备知识 → HTTP / 网络基础（与 Week 01 §4 Q22 同源）。
- Fuzz = 随机/畸形输入找崩；Unit = 已知输入对期望；Property = 随机 + 验性质（`index777 Week 11`，`tb2 Q29`）。Q30：性质测 = 自动生成大量用例，不用逐条手写。

必备知识笔记（不依赖原题）

HTTP / 网络基础

- HTTP 是应用层协议，典型请求-响应模型，且是 stateless（无状态）。
- 常见协议栈顺序：HTTP → TLS → TCP → IP。
- OSI 对应：HTTP → 第 7 层 Application；TCP → 第 4 层 Transport；IP → 第 3 层 Network。
- 以下为 Week 01 §4（tb2 Q22）同款表，在此处复刻以便速查（两处内容一致，改一处时

请同步另一处～。

Q22 选项速判

选项	层	对不对
A 6 Presentation	表示层 / 编码与抽象	x TCP 不在此
B 5 Session	会话层（理论偏多～）	x
C 4 Transport	端到端：端口、可靠/分段	✓ TCP、UDP
D 2 Data Link	链路：帧、MAC、同一段链路	x

OSI 七层一览（职责 + 协议举例：讲义口径；与 Week 01 §4 同源～）

层号	OSI（中英～）	这层做什么（讲义口径～）	常见协议 / 关键词
7	Application 应用层	直接为用户/应用程序 提供可用的网络通信（报文语义、具体应用怎么用～）	HTTP/HTTPS、FTP、DNS ；亦常见 SMTP 等
6	Presentation 表示层	数据格式的转换与协商；加密/解密、编码等（怎么表示～）的问题	TLS （也常画在 TCP 之上的 HTTPS 栈里～）； JPEG 等格式例
5	Session 会话层	管理通信会话：建立、维护、终止连接/对话状态	SOCKS （代理/会话代表例～）；单题考得少
4	Transport 传输层	端到端传输（常理解为	TCP （可靠、面向连接～）、 UDP （无

层号	OSI(中英)	这层做什么(讲义口径)	常见协议 / 关键词
		进程到进程；可靠性、流量控制、端口	连接； tb2 Q22 TCP 在本层 ✓
3	Network 网络层	路由与转发：把分组/数据包投递到对岸网络；逻辑寻址 跨多个网络	IP (v4/v6)； ICMP (常与IP同记)
2	Data Link 数据链路层	在同一段链路/LAN上传 帧；链路节点之间的可检测错/寻址—— MAC	以太网(IEEE 802.3 MAC 子层)； Wi-Fi (802.11 MAC)； MAC 地址
1	Physical 物理层	比特在介质上的电气/光学/射频传输(线上是什么波形)	Ethernet PHY (802.3 物理层)；双绞线/光纤；无线电；蓝牙物理层等

- HTTP 报文基本结构：start-line + headers + 空行 + body (body 可为空)。
- **HEAD** 返回头部元数据，不返回消息体。
- **GET** 通常只读；**POST** 常用于提交数据；**PUT/DELETE** 通常讨论幂等性。幂等速记：**GET / PUT / DELETE** 常判 ✓；**POST** 常判 ✗ (详表：Week 01 → HTTP 方法与幂等性)。实务里 **POST ?action=delete** 替代 **DELETE** (index777 1.7.3)。

URI / Request Target

- URI 常用形态：scheme://host/path?query #fragment (fragment 可选，以 # 开头；见 Week 02 → Q4)。
- #fragment：指向同一资源内部的子部分(如 HTML 锚点)；≠ ?query (参数) ≠

path 层级路径。

- 默认大小写规则：scheme、host 不敏感；path 不应默认不敏感。
- request-line 形式：METHOD request-target HTTP/version。
- request-target 核心是 path (+ query)，用于定位资源与筛选参数——一般不包含 #fragment。fragment 多为客户端滚动/定位。

状态码与响应语义

- 先看首位判大类：1xx/2xx/3xx/4xx/5xx。
- 2xx 是成功类，204 也是成功（只是无内容）。
- 4xx 偏客户端问题（如请求格式、资源不存在）。
- 5xx 偏服务端故障。
- 状态码 + 头部字段一起读，语义更完整（如重定向常配 Location）。

Headers 常用判断

- Host：目标主机。
- User-Agent：客户端环境线索（但不能过度推断）。
- Accept / Accept-Language / Accept-Encoding：客户端可接受格式、语言、压缩。
- Content-Type：消息体类型；Content-Length：消息体字节数。
- Connection: keep-alive：倾向复用连接，不等于“多客户端并发”。

HTML / CSS

- HTML 复习重点是“语义”而非视觉表现（如 h1 的语义作用）。
- 超链接：可点击的呈现文字在 <a>... 内容；href = 目标；title = 补充/悬停（Week 02 Q5）。
- Thymeleaf：服务端模板，Model → 模板 → HTML；@PathVariable 配 /.../{id}，别把字面量 /ids 当成 {id}（resit Q28，Week 02 §5）。
- 选择器高频：#id、.class、后代空格、子代 >、相邻兄弟 +、通用兄弟 ~。
- inline / block 行为差异与 box model 是基础必考（见 §5.1 inline vs block 展开：宽高是否生效、上下 margin 是否参与 collapse、inline-block 的折中行为、 设宽无效等）。
- margin collapse 主要发生在垂直相邻块级元素。
- media query 先判媒体类型（screen/print），再判条件，再看层叠与继承。
- Grid auto-placement 默认按书写方向放置（通常左到右、上到下）。

JavaScript / 异步

- `+` 在字符串参与时可能触发拼接，先确认类型再算结果。
- `let`、`this`、`for...of` 里的 `of` 等是关键字；`not` 不是关键字，否定用 `!`。
- `let` 不能在同一作用域重复声明（与讲义 p.56 里 `var` 允许重复声明对照）；增值写 `curval = curval + 1`，不要再一次 `let curval`。
- `fetch`：收到 HTTP 响应（即使 404/500）通常 fulfilled；网络失败才 rejected。
- `Promise.all`：全部 fulfilled 才进入 `then`；任一 rejected 则整体 rejected。

Scraping / 安全 / 测试

- 抓取规范：优先查 API；遵守 `robots.txt` 与条款；注意再发布与版权；不要信“访问≠下载所以必侵权”这种说法（tb2 Q20）。
- **BeautifulSoup**：`find('x')` 是标签名不是 `id`；`find_all('a')[0]` 仍是 **Tag**；要字符串用 `.text`；`.contents` 是 **list**（tb2 Q21）° 链式 `tag.div.a` = 每层取第一个符合条件的子结点，易跑偏时改用 `select / find(id=...)`。
- `wget -nc`：已有同名文件不覆盖且不重下；`-N` 按时间戳比较。
- 客户端校验主要提升体验，安全性必须由服务端校验兜底。
- 证书链看三项：`subject`、`issuer`、`NotAfter`。
- 邮件“加密+签名”分别对应机密性与身份/完整性验证。
- Testing：**Fuzz**（随机输入探崩溃）vs **Unit**（固定用例）vs **Property**（随机但验不变量）；少手写用例，tb2 Q30；见 **Week 11**（index777 p.97-103，tb2 Q29 Q30）。